

JAMSEC

Servicios Gestionados de Ciberseguridad

MEMORIA DEL PROYECTO

Infraestructura empresarial, monitorización y auditoría de seguridad

	JAMSEC / Apex Gestoría
Autor	Alex Sánchez García
Curso	Especialización en Ciberseguridad
Fecha	Junio de 2026
Plataforma	Clúster Proxmox VE 9.1 (3 nodos físicos)

1. Introducción

La idea de este proyecto es ponerme en la piel de una empresa de ciberseguridad de verdad. Esa empresa es JAMSEC, y a lo largo del proyecto monta su propia infraestructura, vigila la de un cliente y, cuando este lo contrata, le hace una auditoría para ver hasta dónde podría llegar un atacante. El cliente es una gestoría ficticia, Apex Gestoría, que es la que voy a acabar atacando.

He querido que no fuera un montaje «de juguete». En vez de simularlo todo en una herramienta de redes, lo he levantado sobre un clúster de virtualización Proxmox de tres nodos físicos reales, con máquinas virtuales de pleno derecho, servicios funcionando y tráfico real entre ellas. La red de tránsito que comparto con el aula tiene su propia complejidad: otras máquinas también la usan, lo que me obligó a gestionar bien el direccionamiento y a aprender a detectar conflictos de IP en caliente. Creo que ese tipo de «fricción real» es lo que más se acerca a trabajar en una empresa.

Otra cosa que me marqué desde el principio fue no configurar nada «a mano y a lo loco». Casi todo está hecho con cloud-init, lo que significa que cada máquina se auto-configura al arrancar: hostname, usuarios, claves SSH, paquetes instalados... Si mañana se borra una VM, la levanto de nuevo en minutos con el mismo resultado. Además, cualquier tarea que hay que hacer varias veces la he convertido en un script que queda guardado, documentado y subido al repositorio.

Por una norma del entorno compartido, en ningún momento he reiniciado ni apagado los nodos físicos del clúster. Todo lo he resuelto recargando servicios o reiniciando solo las máquinas virtuales que son mías. Eso me obligó a ser más cuidadoso y a entender bien qué depende de qué para no dejar nada colgado.

1.1. Los actores del proyecto

Para entender el proyecto es importante tener claro quién es quién:

JAMSEC S.L.	La empresa de ciberseguridad. Tiene su propia infraestructura (SOC, web, ficheros) y es desde donde se realiza la monitorización y la auditoría.
Apex Gestoría	El cliente. Una gestoría con datos fiscales confidenciales, su propia web, su base de datos y un servidor de ficheros. Es la víctima de la auditoría.
Clúster Proxmox	La «sala de máquinas» donde vive todo. Tres nodos físicos (proxmox, pve2, pve3) que forman un único clúster con gestión centralizada.
Alex (yo)	El técnico de JAMSEC. Monto la infraestructura, la monitorizo y, cuando llega el momento, me pongo el sombrero de atacante para auditar a Apex.

1.2. Contexto y motivación

El proyecto nace de una premisa sencilla: los laboratorios de clase con redes simuladas están bien para aprender los conceptos, pero no preparan para lo que te encuentras al salir. En un entorno real hay máquinas que fallan por razones ajenas a ti, redes que comparten otras personas, servicios que se niegan a arrancar sin explicación y documentación que hay que escribir sobre la marcha. Quería que este proyecto tuviera todo eso.

El escenario de una gestoría tampoco es casual. Las gestorías manejan datos muy sensibles (nóminas, declaraciones fiscales, contratos) pero normalmente no tienen un departamento de IT ni presupuesto para seguridad. Son exactamente el tipo de empresa que cae con ataques que no son sofisticados: contraseñas por defecto, webs sin actualizar, FTP abierto. Con Apex Gestoría quería recrear eso de forma realista, no plantarle un servidor expuesto con contraseña «1234» sin sentido.

2. Objetivos

Al arrancar el proyecto me propuse una serie de objetivos concretos. Los pongo aquí junto con una nota de por qué cada uno estaba en la lista:

- Montar una infraestructura empresarial propia, con cortafuegos, zonas de red separadas (WAN, DMZ, SOC, servicios internos) y servicios reales funcionando.
- Montar, de forma completamente separada, la infraestructura del cliente Apex Gestoría, con su propia topología y sus vulnerabilidades plantadas.
- Implantar un sistema IDS/SIEM que cubra las dos infraestructuras a la vez y permita ver lo que ocurre desde un único panel.
- Realizar una auditoría de penetración completa contra Apex Gestoría, cubriendo al menos cuatro vectores distintos: web, red, servicios y contraseñas.
- Redactar un informe técnico con todos los hallazgos, las pruebas de explotación y las recomendaciones de remediación.
- Redactar un informe ejecutivo que un cliente sin perfil técnico pueda entender, centrado en el impacto de negocio y no en el detalle técnico.
- Automatizar el despliegue con cloud-init y scripts para que todo se pueda reproducir sin tener que recordar comandos de memoria.
- Documentar el proyecto y publicar el resultado en un repositorio GitHub público, con los informes en PDF y los recursos necesarios.
- Preparar una demostración en vivo de la auditoría para el tribunal, reproducible en el entorno real.

Para no perderme, los fui haciendo en este orden y no empezaba uno hasta tener el anterior funcionando y probado. Me costó más tiempo así, pero me evité el típico error de montar cosas encima de otras que todavía no iban bien.

3. Marco tecnológico

Antes de entrar en el detalle de qué monté, viene bien explicar qué herramientas elegí y por qué. En ningún caso fue una decisión aleatoria: cada herramienta tiene un motivo detrás.

3.1. Proxmox VE como hipervisor

Proxmox Virtual Environment es un hipervisor de tipo 1 basado en KVM y LXC que corre directamente sobre el hardware, sin necesidad de un sistema operativo anfitrión. Lo elegí porque el entorno del aula ya lo tiene instalado, porque permite montar un clúster con varios nodos físicos gestionados desde una sola interfaz web, y porque da acceso a funciones de empresa (backups, snapshots, migración en caliente) sin coste de licencia.

El hecho de que sea un clúster de tres nodos no es solo cosmético: significa que puedo distribuir las máquinas virtuales entre nodos distintos y que el clúster comparte el estado (qué máquinas hay, en qué nodo están, qué recursos usan) entre los tres. En la práctica lo noté cuando tuve que hablar con máquinas que estaban en «pve2» desde «proxmox»: son nodos distintos pero el clúster los trata como una sola unidad.

3.2. Debian 12 y 13 como sistema operativo base

Prácticamente todas las máquinas virtuales corren sobre Debian. Elegí Debian por su estabilidad, su ciclo de soporte largo y porque es la distribución que mejor conozco. Usé Debian 12 (Bookworm) para la mayoría de los servicios y Debian 13 (Trixie, en su versión testing en el momento de montarlo) para el servidor Wazuh, porque la plantilla cloud-init que había disponible en el clúster era de esa versión. Eso me generó algún problema de compatibilidad, que cuento en la sección de problemas.

3.3. nftables como cortafuegos

Para los cortafuegos de ambas infraestructuras usé nftables, el sucesor de iptables en el kernel de Linux. nftables tiene una sintaxis más limpia, agrupa todas las reglas en un único archivo de configuración y es el estándar que trae Debian por defecto. La alternativa habría sido usar pfSense o OPNsense (appliance de firewall dedicados), pero no había plantilla disponible en el clúster y, siendo sincero, prefería montar el firewall a mano para entender de verdad cómo se filtra el tráfico entre zonas.

La configuración que hice fue en tres partes: reglas de entrada (qué llega desde fuera), reglas de salida (qué pueden mandar las máquinas internas) y reglas de reenvío (qué pasa de una zona a otra), más las reglas de NAT para que las máquinas sin IP pública puedan salir a internet.

3.4. Wazuh como SIEM

Wazuh es una plataforma de seguridad de código abierto que combina detección de intrusiones en el host (HIDS), gestión de logs, monitorización de la integridad de ficheros y análisis de vulnerabilidades. Está formada por un servidor central que recibe los datos y los correlaciona, y agentes ligeros que se instalan en cada máquina vigilada y le mandan los eventos.

Lo elegí porque es el SIEM gratuito más completo que existe y porque en clase lo habíamos visto, así que tenía una base. En la versión 4.12 que usé ya viene con un dashboard propio basado en OpenSearch, lo que significa que no hay que montar Elasticsearch y Kibana por separado como en versiones anteriores. Esto simplificó bastante la instalación.

3.5. Suricata como IDS de red

Suricata es un motor de detección de intrusiones de red (NIDS) que analiza el tráfico en tiempo real y lo compara con una base de reglas. Lo coloqué en los cortafuegos de ambas infraestructuras, que son el punto por donde pasa todo el tráfico hacia y desde internet. Usando el modo af-packet, puede capturar paquetes directamente desde la tarjeta de red sin perder rendimiento.

Las reglas que usé son las de Emerging Threats Open (ET Open), que son gratuitas y cubren una cantidad enorme de patrones: escaneos de puertos, exploits conocidos, tráfico de malware, intentos de fuerza bruta... Sus alertas las mandé a Wazuh a través del log de alertas de Suricata, así que aparecen en el mismo panel que los eventos del sistema.

3.6. Tailscale como VPN de acceso remoto

Tailscale es una solución de VPN basada en WireGuard que permite conectar máquinas de forma segura sin necesidad de abrir puertos en el router ni tener IP pública. Funciona creando una red virtual privada entre todos los nodos donde está instalado. Lo usé para acceder al clúster Proxmox desde fuera de la red del aula sin depender de la IP pública del centro.

La instalé en los nodos físicos del clúster (proxmox y pve2), lo que me permite conectarme directamente a las interfaces de gestión de Proxmox y, desde ahí, saltar a cualquier máquina

virtual mediante SSH con saltos encadenados (ProxyJump).

3.7. Cloud-init para el despliegue automático

Cloud-init es el estándar de facto para la configuración inicial de máquinas virtuales en la nube. Permite definir en un fichero YAML qué usuario crear, qué clave pública instalar, qué hostname poner y qué paquetes instalar, y la máquina se auto-configura al primer arranque sin intervención manual. En Proxmox funciona nativo con las plantillas que lo soporten.

En la práctica, esto significa que levantar una nueva máquina me lleva tres pasos: clonar la plantilla, rellenar los campos de cloud-init (hostname, IP, clave SSH) y arrancar. La máquina queda configurada y accesible por SSH en menos de dos minutos. Eso me ahorró muchísimo tiempo durante el desarrollo y me garantiza que todas las máquinas tienen la misma base.

4. Infraestructura de JAMSEC

La infraestructura propia de JAMSEC vive en el nodo «proxmox» del clúster. La diseñé pensando en cómo funcionaría una empresa de ciberseguridad pequeña pero real: necesita su propio perímetro, un SOC desde donde operar, una web pública y almacenamiento interno.

4.1. Topología de red

Dividí la infraestructura en cuatro zonas con sus propias redes, cada una con un propósito claro y reglas de tráfico distintas:

		
WAN	172.20.17.0/24	La red de tránsito compartida con el aula. El cortafuegos tiene aquí su pata exterior y es por donde sale el tráfico a internet.
SOC	10.20.10.0/24	El centro de operaciones. Aquí están la máquina de ataque/análisis (jamsec-redteam) y el servidor Wazuh. Solo accesible desde dentro.
DMZ	10.20.20.0/24	La zona desmilitarizada. Aquí está la web corporativa de JAMSEC, que sí se puede ver desde internet (con DNAT en el cortafuegos).
Servicios	10.20.30.0/24	La red interna de servicios. Aquí está el servidor de ficheros Samba, solo accesible desde el SOC.

La separación en zonas no es decorativa. La verifiqué de forma práctica: desde la DMZ intenté llegar a la red de servicios y al SOC, y no pudo. Desde el SOC sí puedo llegar a todas las zonas, porque es desde donde trabajo. Desde internet solo se puede ver la web de la DMZ, y únicamente por los puertos que yo abrí expresamente.

4.2. El cortafuegos (jamsec-fw)

El corazón de la infraestructura es jamsec-fw, una máquina Debian con cuatro tarjetas de red virtuales: una en cada zona. Actúa de router, de cortafuegos y de gateway para todas las máquinas de la red. La configuré con nftables y apliqué una política de denegación por defecto: todo lo que no esté explícitamente permitido se descarta.

Las reglas principales que apliqué son estas:

- La red del SOC tiene acceso completo a internet y a todas las zonas internas. Al fin y al cabo, es donde trabajo yo.
- La DMZ puede salir a internet (para actualizarse), pero no puede hablar con el SOC ni con la red de servicios.
- La red de servicios tampoco puede hablar con la DMZ ni con el SOC por iniciativa propia.
- Desde internet, solo entra lo que el DNAT reenvía expresamente: el puerto 80 hacia la web de la DMZ.
- El tráfico del agente Wazuh hacia el servidor de monitorización está permitido desde todas las zonas, usando la IP de la interfaz de tránsito del servidor Wazuh.

También configuré el reenvío de paquetes IP (ip_forward) y el NAT de salida (masquerade) para que las máquinas sin IP pública puedan actualizar paquetes y descargar herramientas.

4.3. La web corporativa (jamsec-web)

La web de JAMSEC es una página estática servida con nginx. Es lo único de toda la infraestructura que se publica hacia internet (mediante una regla DNAT en el cortafuegos que redirige el puerto 80 de la IP de tránsito hacia esta máquina). El contenido es una landing page corporativa que explica qué es JAMSEC y qué servicios ofrece.

La puse en la DMZ a propósito, y no en el SOC, para demostrar el principio de separación: si alguien encontrara un fallo en la web y la comprometiera, estaría dentro de la DMZ pero no podría llegar al SOC ni a los ficheros internos.

4.4. El servidor de ficheros (jamsec-srv)

En la red de servicios internos monté un servidor Samba que simula un recurso compartido de empresa: donde los empleados guardan documentación interna. Es accesible solo desde el SOC. No tiene nada que ver con la auditoría al cliente, pero lo incluí para que la infraestructura fuera más completa y realista.

4.5. La estación de operaciones (jamsec-redteam)

Dentro del SOC, la máquina más importante para la auditoría es jamsec-redteam. Es una Debian con todas las herramientas de análisis y ataque instaladas: nmap, sqlmap, hydra, nikto, gobuster, netcat, curl... Es desde donde hago todo el trabajo de reconocimiento y explotación contra Apex Gestoría.

La razón de tenerla dentro del SOC de JAMSEC (en vez de atacar directamente desde mi PC) es realista: en un pentest real, la empresa de seguridad usa sus propias máquinas, que están dentro de su perímetro y son de confianza. Atacar desde una máquina externa no controlada sería mala práctica.

4.6. El servidor Wazuh (jamsec-wazuh)

El servidor de monitorización lo monté en el SOC. Además de su tarjeta en la red del SOC, le puse una segunda tarjeta de red en la red de tránsito (172.20.17.x). Esta segunda interfaz es la que usan los agentes de las máquinas de Apex Gestoría para comunicarse con el servidor, ya que no pueden llegar al SOC de JAMSEC por el camino normal sin abrir agujeros en el cortafuegos. Me gustó esta solución porque no rompe la separación de zonas: el agente sale por la IP de tránsito del servidor Wazuh, que sí está en una red que los agentes pueden alcanzar.

En total, el servidor Wazuh gestiona siete agentes: los cuatro de la infraestructura de JAMSEC y los tres de Apex Gestoría que tiene sentido monitorizar. El dashboard propio de Wazuh 4.12 permite ver en tiempo real los eventos de todos ellos, buscar por máquina, por tipo de alerta o por periodo de tiempo.

5. Infraestructura de Apex Gestoría

La infraestructura del cliente la monté en un nodo físico distinto (pve2) a propósito. Quería que se notara que son dos empresas separadas de verdad y no dos redes apañadas en la misma máquina. Apex tiene su propio cortafuegos, su DMZ con la web pública y una red interna donde vive la base de datos y el servidor de ficheros.

5.1. Topología de red de Apex

La topología de Apex es más sencilla que la de JAMSEC porque refleja lo que suele tener una pyme que no ha invertido mucho en seguridad: dos zonas (DMZ y LAN) con un cortafuegos en el perímetro.

WAN	172.20.17.0/24	La pata exterior del cortafuegos. IP pública: 172.20.17.230.
DMZ Apex	10.30.20.0/24	apex-web (10.30.20.10): el servidor web con la aplicación PHP.
LAN Apex	10.30.10.0/24	apex-db (10.30.10.10): MariaDB. apex-srv (10.30.10.20): FTP y SSH internos.

5.2. El cortafuegos de Apex (apex-fw)

El cortafuegos de Apex está configurado de forma deliberadamente débil, para representar lo que hace una pyme que «algo» ha configurado pero sin demasiado rigor. Tiene las reglas básicas de NAT y unas reglas de DNAT para publicar los servicios, pero la separación entre la DMZ y la LAN es casi inexistente: apex-web puede llegar a apex-db y apex-srv sin pasar por ningún filtro, como si fueran la misma red.

Las reglas de DNAT que configuré, que son las que exponen servicios hacia internet, son:

- Puerto 80 (HTTP) → apex-web: la web pública de la gestoría.
- Puerto 21 (FTP) → apex-srv: el servidor de FTP con los ficheros de la gestoría.

- Puerto 2222 (SSH alternativo) → apex-web puerto 22: acceso SSH al servidor web.

5.3. El servidor web (apex-web)

El servidor web corre Apache 2.4 con PHP 8.2 sobre Debian. La aplicación web es una gestión de empleados y clientes de una gestoría ficticia: tiene una página de inicio, un formulario de login y un panel de empleados que muestra datos de clientes y permite subir ficheros.

La aplicación la escribí yo desde cero con dos vulnerabilidades intencionales muy bien documentadas en el OWASP Top 10:

Inyección SQL en el login (A03:2021). El campo de usuario del formulario de login se concatena directamente en la consulta SQL sin ningún tipo de escape ni prepared statement. Un payload como `admin' OR '1'='1'--` - como nombre de usuario hace que la consulta devuelva siempre verdadero y permite entrar sin contraseña.

Subida de ficheros sin validación (A01:2021). El panel de empleados permite subir ficheros sin comprobar en ningún momento el tipo, la extensión ni el contenido. Eso significa que puedo subir un script PHP disfrazado de cualquier cosa, y el servidor lo ejecutará si accedo a él por HTTP. Es la definición de una webshell.

5.4. La base de datos (apex-db)

Apex-db corre MariaDB 10.11. Contiene la base de datos apexgestoria con cuatro tablas: usuarios (credenciales de acceso a la web), clientes (nombre, NIF, correo electrónico e IBAN de doce clientes), nominas (trabajador, DNI, salario bruto e IBAN de ocho empleados) y facturas (doce facturas con importes y estados de pago).

Los datos que puse son ficticios pero realistas: nombres de empresas catalanas, NIFs y NIF con formato válido, IBANs bien formados. Quería que el impacto de la exfiltración de datos se entendiera de verdad: no es «saqué una tabla vacía», es «saqué los salarios de todos los empleados y los IBANs de todos los clientes».

Las vulnerabilidades de configuración que tiene son dos: el usuario root tiene una contraseña ridícula (toor), y el servidor escucha en todas las interfaces de red, no solo en localhost. Eso significa que cualquier máquina que pueda alcanzar apex-db puede intentar conectarse a la base de datos directamente.

5.5. El servidor de ficheros (apex-srv)

Apex-srv simula el servidor de ficheros interno de la gestoría. Tiene dos servicios expuestos con vulnerabilidades:

- FTP con acceso anónimo (vsFTPD 3.0.5): cualquiera puede conectarse sin usuario y descargarse los ficheros del directorio público. En ese directorio dejó dos ficheros: un backup parcial de la base de datos SQL y un fichero de credenciales en texto claro que «alguien olvidó» allí. También hay un directorio incoming con permisos 777, es decir, cualquier anónimo puede subir lo que quiera.
- SSH con contraseña débil: el usuario backup tiene contraseña backup2024. Este usuario existe para que el departamento de IT pueda conectarse al servidor de forma remota, pero la contraseña no cumple ninguna política mínima de seguridad.

5.6. Por qué estos fallos concretos

No planté estos fallos al azar. Cada uno representa una categoría de error real que me encontraría en una pyme:

SQLi en login		Es la vulnerabilidad más encontrada en aplicaciones web a medida. Ocurre cuando el desarrollador no conoce los prepared statements.
Subida sin validar		Muy común en webs con funcionalidad de adjuntos. El desarrollador solo piensa en «que el usuario pueda subir el fichero», no en qué puede hacer el atacante con eso.
FTP anónimo		Sorprendentemente frecuente. El FTP se configura una vez, nadie lo revisa, y acaba teniendo el acceso anónimo habilitado por defecto.
Contraseña SSH	débil	Clásico en cuentas de servicio. Se crean con una contraseña «temporal» que nunca se cambia.
Red LAN	plana DMZ-	El fallo más difícil de ver desde dentro. Si nadie ha documentado para qué sirve cada VLAN, nadie las segmenta bien.
Credenciales en texto claro	en	El fichero credenciales.txt en el FTP es algo que pasa: alguien lo subió para compartirlo con un compañero y se olvidó de borrarlo.

6. Monitorización: IDS/SIEM

La parte de monitorización es, junto con la auditoría, la que más tiempo me llevó. La idea era que el sistema de vigilancia estuviera funcionando mientras hacía la auditoría, de forma que pudiese ver en tiempo real cómo el ataque quedaba registrado en el SIEM. Eso daría sentido a tener las dos piezas (ataque y defensa) en el mismo proyecto.

6.1. Wazuh: instalación y configuración

Instalé Wazuh 4.12 usando el asistente oficial, que instala el servidor, el indexador (OpenSearch) y el dashboard en la misma máquina. Lo hice en Debian 13, y ahí me topé con el primer problema grande: el asistente intentaba instalar un paquete llamado debconf-utils que ya no existe en Debian 13 con ese nombre. Tuve que editar el script del instalador para quitar esa dependencia, lo que me llevó un buen rato porque el error no aparecía de forma obvia.

Una vez instalado, le di 4 GB de RAM a la máquina (OpenSearch consume bastante) y configuré los certificados para HTTPS. El dashboard quedó accesible en el puerto 443 con autenticación.

6.2. Agentes Wazuh en todas las máquinas

Instalé el agente de Wazuh en siete máquinas: los cuatro nodos de JAMSEC que tienen sentido monitorizar (fw, web, srv y redteam) y tres de Apex (fw, web y db). No puse agente en apex-srv porque es la víctima directa del FTP, y quería que fuera «ciega» a lo que pasa, para que el contraste con la detección del SIEM fuera más claro.

El truco de la doble interfaz del servidor Wazuh (una en el SOC y otra en la red de tránsito) fue clave aquí. Los agentes de las máquinas de Apex se registran usando la IP de tránsito del servidor Wazuh (172.20.17.231), que sí pueden alcanzar. Los agentes de JAMSEC usan la IP del SOC (10.20.10.x). Así, ningún agente necesita que se abran agujeros en los cortafuegos: cada uno sale por la interfaz que tiene disponible.

Hubo un detalle que me dio guerra: la versión del agente que se descarga por defecto era más nueva que el servidor que había instalado, y Wazuh no acepta agentes de versión superior. Tuve que fijar la versión del agente explícitamente en el comando de instalación.

6.3. Suricata como IDS de red

Instalé Suricata en los cortafuegos de ambas infraestructuras (jamsec-fw y apex-fw). Lo configuré en modo af-packet sobre la interfaz de WAN, de forma que ve todo el tráfico que entra y sale. Las reglas ET Open las actué con suricata-update, que se encarga de descargarlas y mantenerlas actualizadas.

Los dos problemas que tuve con Suricata los cuento en la sección de problemas, pero resumiendo: uno de los cortafuegos se quedaba sin memoria al arrancar Suricata (lo resolví dándole más RAM a la VM) y en ambos aparecían miles de falsas alarmas relacionadas con la tarjeta de red virtual (las resolví desactivando el offloading de la NIC y limitando Suricata a un solo hilo).

La integración con Wazuh es automática si se configura la ruta del log de alertas de Suricata como fuente de logs en el agente Wazuh. Una vez hecho eso, las alertas de red de Suricata aparecen en el dashboard de Wazuh junto con los eventos del sistema.

6.4. Qué detectó el SIEM durante la auditoría

Lo que más me satisfizo fue comprobar que el SIEM detectó el ataque mientras lo hacía. No todo, y no siempre con el nivel de detalle que me gustaría, pero sí lo suficiente como para que un analista de guardia hubiera podido verlo en tiempo real:

- El escaneo de nmap generó alertas de Suricata en apex-fw por reconocimiento de puertos (reglas de la categoría ET SCAN).
- Los intentos de login fallidos del formulario web aparecieron como intentos de autenticación en los logs de Apache, recogidos por el agente de apex-web.
- La fuerza bruta de Hydra contra el SSH generó múltiples alertas de autenticación fallida en apex-web, que Wazuh agrupa automáticamente en una alerta de fuerza bruta cuando supera el umbral.
- La subida de la webshell apareció en los logs de Apache como un POST al panel de subida de ficheros seguido de un GET a uploads/shell.php.
- Las consultas a la base de datos desde la webshell generaron tráfico que Suricata detectó como conexiones SQL anómalas entre la DMZ y la LAN.

7. La auditoría de seguridad

Con todo montado, me puse el sombrero de atacante. Desde jamsec-redteam, dentro del SOC, hice una auditoría de caja negra: empecé sin saber nada de la infraestructura de Apex, igual que un atacante de fuera. Para organizarme seguí el esquema de PTES (Penetration Testing Execution Standard), que divide el trabajo en fases, y para la parte web usé el OWASP Top 10 como referencia de qué buscar. La puntuación de gravedad de cada hallazgo la calculé con CVSS 3.1.

7.1. Fase 1 — Reconocimiento

Lo primero, antes de intentar nada, es saber qué hay. Hice un escaneo completo de puertos con nmap contra la IP pública de Apex (172.20.17.230):

```
nmap -sV -sC -p- --open -T4 172.20.17.230
```

El resultado fue claro y bastante preocupante para el cliente: cuatro puertos abiertos con sus servicios identificados.

21/tcp	FTP	vsFTPD 3.0.5. nmap detectó automáticamente que permite acceso anónimo.
22/tcp	SSH	OpenSSH 10.0p2. El SSH del propio cortafuegos.
80/tcp	HTTP	Apache 2.4.67. Cabecera Server visible, lo que revela la versión exacta.
2222/tcp	SSH	OpenSSH 10.0p2. Un segundo SSH en puerto no estándar (DNAT hacia apex-web).

Solo con este escaneo ya hay dos hallazgos importantes: el FTP anónimo y la versión de Apache visible. El resto del trabajo de reconocimiento fue analizar la web con nikto y whatweb para detectar tecnologías y posibles configuraciones erróneas.

7.2. Fase 2 — Análisis de la aplicación web

Con Apache y PHP identificados, me centré en la aplicación web. Usé whatweb para fingerprinting automático y nikto para detectar configuraciones erróneas habituales. Lo más relevante que encontré en esta fase fue que el directorio uploads/ era accesible directamente por HTTP (sin ningún control de acceso), y que el formulario de login no tenía ninguna protección anti-automatización (sin CAPTCHA, sin límite de intentos).

También hice una exploración manual de la web: el formulario de login en login.php y el panel en panel.php, que requería sesión activa. Eso me dio el mapa de la aplicación antes de empezar a atacarla.

7.3. Fase 3 — Inyección SQL

La vulnerabilidad más grave que encontré en la web fue la inyección SQL en el formulario de login. Lo detecté de forma manual primero, probando el payload clásico de bypass de autenticación:

Usuario: admin' OR '1'='1'-- - Contraseña: cualquiera

La respuesta del servidor fue un HTTP 302 hacia panel.php, lo que significa que la autenticación se saltó completamente. Con ese resultado confirmado, lancé sqlmap para automatizar la explotación:

```
sqlmap -u "http://172.20.17.230/login.php" --data="usuario=admin&password=test" --dbms=mysql --code=302 --dbs --batch
```

sqlmap encontró tres tipos de inyección distintos (booleana ciega, basada en tiempo y UNION) y enumeró las bases de datos disponibles. La base de datos apexgestoria tenía cuatro tablas: usuarios, clientes, nominas y facturas. El volcado completo de esas tablas me dio acceso a todos los datos de la gestoría: credenciales de acceso a la web, NIF e IBAN de doce clientes, nóminas de ocho empleados con sus DNI y salarios, y el historial de facturas.

7.4. Fase 4 — Webshell y ejecución remota de código

Una vez dentro del panel (usando la SQLi para autenticarme), exploré la funcionalidad de subida de ficheros. El formulario permitía subir cualquier fichero sin ninguna restricción: ni por extensión, ni por tipo MIME real, ni por contenido. Subí una webshell PHP de una sola línea:

```
<?php if(isset($_GET['cmd']))){echo '<pre>'.shell_exec($_GET['cmd']).' </pre>';}>
```

El fichero quedó accesible en <http://172.20.17.230/uploads/shell.php>. Desde ahí pude ejecutar cualquier comando del sistema como el usuario `www-data`:

- Confirmé el usuario (`www-data`) y el hostname (`apex-web`).
- Leí el fichero `/etc/passwd` para enumerar los usuarios del sistema.
- Listé los ficheros de la aplicación web para encontrar más información de configuración.

Este hallazgo es especialmente grave porque `www-data` tiene permisos de escritura en el directorio de uploads y de lectura en los ficheros de configuración de la aplicación, donde estaban las credenciales de conexión a la base de datos.

7.5. Fase 5 — Exfiltración de base de datos vía webshell

Con la webshell activa intenté conectarme a la base de datos directamente. El cliente MySQL no estaba instalado en `apex-web`, pero PHP tiene el módulo `mysqli` nativo. Subí un script PHP que usaba `mysqli` para conectarse a `apex-db` (10.30.10.10) con las credenciales `root/toor` (encontradas más tarde en el FTP) y volcar todas las tablas. El resultado fue el volcado completo de los datos de la gestoría, accesible desde internet por HTTP.

7.6. Fase 6 — FTP anónimo

Con el puerto 21 identificado en el reconocimiento inicial, probé la conexión FTP con el usuario `anonymous`:

```
ftp -n 172.20.17.230 → user anonymous anonymous@audit
```

El login fue exitoso sin ninguna autenticación. En el directorio raíz encontré tres elementos:

- `backup_apexgestoria.sql`: un backup parcial de la base de datos con las credenciales de los usuarios de la web en texto claro.
- `credenciales.txt`: un fichero con las contraseñas de MariaDB (`root/toor`), de la aplicación web (`apexweb/apexweb`), del router/firewall (`admin/admin`) y de la WiFi de la oficina (`ApexGestoria2024`).
- Directorio `incoming/` con permisos `777`: cualquier usuario anónimo puede subir ficheros al servidor.

El hallazgo del fichero de credenciales es extraordinariamente grave: con esa información un atacante tiene acceso a prácticamente todos los sistemas de la empresa sin necesidad de explotar nada más.

7.7. Fase 7 — Fuerza bruta SSH

Con el puerto 2222 identificado como SSH hacia apex-web, lancé Hydra con un diccionario pequeño de usuarios y contraseñas típicas de gestorías:

```
hydra -L users.txt -P pass.txt -s 2222 -t 4 -V 172.20.17.230 ssh
```

En menos de treinta segundos Hydra encontró la credencial soporte / soporte123. Con esa credencial me conecté al servidor web por SSH y obtuve una shell interactiva como el usuario soporte, con acceso a la carpeta home y a la configuración de la aplicación web.

7.8. Fase 8 — Pivote DMZ → LAN

Esta fue la fase más importante desde el punto de vista de impacto. El objetivo era demostrar que un atacante que compromete la DMZ (apex-web) puede llegar a la red interna de la empresa (donde están apex-db y apex-srv) sin que ningún control lo impida.

Lo demostré de tres formas distintas:

- Desde la webshell (www-data en apex-web): pings con 0% de pérdida a apex-db (10.30.10.10) y apex-srv (10.30.10.20). La red interna es completamente alcanzable desde la DMZ.
- Desde la webshell (PHP mysqli): conexión directa al puerto 3306 de apex-db y volcado de toda la base de datos. Acceso completo a los datos más sensibles de la empresa desde internet.
- Desde soporte@apex-web: intento de conexión SSH a apex-srv con las credenciales backup/backup2024. La autenticación fue exitosa (el servidor aceptó la contraseña), aunque la cuenta tiene el shell configurado como nologin para bloquear el acceso interactivo. Eso demuestra que las credenciales son válidas y que la red interna es alcanzable, aunque este control concreto limita lo que se puede hacer una vez autenticado.

La conclusión de esta fase es clara: si la DMZ y la LAN estuvieran correctamente segmentadas (con el cortafuegos filtrando qué puede pasar de una a otra), ninguna de estas tres vías funcionaría. La red plana es el fallo que hace que todos los demás fallos sean mucho más graves.

8. Resumen de hallazgos

Al terminar la auditoría, clasifiqué los hallazgos según CVSS 3.1 y los agrupé por gravedad. En total salieron nueve vulnerabilidades distintas: tres críticas, tres altas y tres medias.

	Vulnerabilidades					
1	Inyección SQL en login.php	9.8 Crítico	sqlmap / curl	Acceso sin autenticación + volcado completo de la BBDD.		
2	Subida sin validar → Webshell / RCE	9.0 Crítico	curl / PHP	Ejecución arbitraria de comandos en el servidor web.		
3	Red plana — DMZ y LAN sin segmentar	8.1 Alto	nmap / ping	Permite el pivote desde la DMZ comprometida hasta la LAN.		
4	FTP anónimo con credenciales expuestas	7.5 Alto	ftp	Contraseñas de BBDD, WiFi y router accesibles sin autenticación.		
5	SSH con contraseñas débiles	7.5 Alto	Hydra	Acceso interactivo al servidor web como usuario soporte.		
6	Cabeceras HTTP que revelan versiones	5.3 Medio	nmap / nikto	El atacante sabe qué versión exacta de Apache y PHP buscar en CVEs.		
7	Directorio uploads/ accesible sin control de acceso	5.3 Medio	navegador	Permite acceder a cualquier fichero subido, incluidas webshells.		
8	MariaDB escucha en todas las interfaces	5.3 Medio	nmap	El puerto 3306 es alcanzable desde cualquier máquina de la red.		
9	Sin monitorización ni logging en Apex	4.9 Medio	—	Los ataques no dejan ningún rastro visible para el cliente.		

El detalle completo de cada hallazgo —descripción técnica, prueba de explotación, CVSS desglosado y pasos de remediación— está en el informe técnico. El informe ejecutivo recoge el mismo contenido pero sin la parte técnica y con el foco en el impacto de negocio.

9. Problemas encontrados y cómo los resolví

No todo salió a la primera. Esta sección recoge los problemas más importantes que me encontré a lo largo del proyecto, porque creo que son tan parte del aprendizaje como lo que funcionó bien.

Conflictos de IP en la red de tránsito. Otras máquinas del aula usaban las mismas IPs que yo y dejaban de responder de repente.	Aprendí a comprobar siempre con ping y con la tabla ARP (<code>arp -n</code>) si una IP estaba libre antes de asignarla. Reasigné las conflictivas a rangos menos usados.
El instalador de Wazuh 4.12 fallaba en Debian 13 con un error sobre el paquete <code>debconf-utils</code> que ya no existe en esa versión.	Edité el script del instalador para eliminar esa dependencia. Me costó encontrarlo porque el error aparecía en medio de un log muy largo y no era nada descriptivo.
Los agentes de Wazuh recién instalados (versión más nueva) eran rechazados por el servidor con versión ligeramente inferior.	Fijé la versión del agente en el comando de instalación para que coincidiera exactamente con la del servidor. Wazuh no acepta agentes más nuevos que el manager.
Suricata generaba miles de falsas alarmas relacionadas con la tarjeta de red virtual (<code>checksum errors</code> , <code>offload alerts</code>).	Desactivé el offloading de la NIC con <code>ethtool</code> y configuré Suricata para usar un solo hilo. Las falsas alarmas desaparecieron completamente.
Uno de los cortafuegos se quedaba sin memoria al intentar arrancar Suricata y el servicio no levantaba.	Le di más RAM a la VM desde Proxmox. A los nodos físicos no podía tocarlos, pero la VM es mía y puedo cambiar sus recursos sin reiniciar el nodo.
sqlmap no detectaba la inyección en el primer intento con las opciones por defecto.	El problema era que la inyección exitosa da un HTTP 302 (redirección al panel), no un HTTP 200. Añadí <code>--code=302</code> para que sqlmap supiera que ese era el indicador de éxito.
La VPN Tailscale del nodo pve2 dejó de responder durante el proyecto, cortando el acceso remoto por ese camino.	El nodo proxmox tenía Tailscale activo con otra IP. Construí una cadena de ProxyJump alternativa que va de proxmox a pve2 por la red LAN del clúster y de ahí a las máquinas virtuales.

El cliente MySQL no estaba instalado en apex-web, lo que impedía volcar la base de datos desde la webshell.

PHP tiene el módulo mysqli nativo y no necesita el cliente de línea de comandos. Subí un script PHP que conectaba directamente a apex-db con mysqli y volcaba las tablas por HTTP.

10. Planificación temporal

El proyecto lo dividí en fases que fui completando de forma secuencial. La estimación de horas es aproximada porque hubo días con más avance y días donde un problema concreto me consumió más tiempo del previsto.

1	Diseño de la arquitectura y planificación de redes	8 h	Diagramas y plan de direccionamiento
2	Montaje de la infraestructura de JAMSEC (fw, web, srv, redteam)	20 h	Cuatro VMs operativas
3	Montaje de la infraestructura de Apex (fw, web, db, srv)	18 h	Cuatro VMs operativas con vulns
4	Instalación y configuración de Wazuh + agentes	16 h	SIEM con 7 agentes activos
5	Instalación y ajuste de Suricata en ambos fw	10 h	IDS activo en perímetros
6	Auditoría de penetración completa contra Apex	20 h	Todas las fases completadas
7	Redacción del informe técnico y ejecutivo	14 h	Dos informes en PDF
8	Documentación, repositorio, presentación y memoria	12 h	Repo público + PPTX + esta memoria
Total		~118 h	

11. Valoración económica

Una parte del proyecto era estimar cuánto costaría montar algo así en la realidad. Hice dos estimaciones: lo que ha costado en el entorno académico (casi nada, porque el hardware estaba disponible) y lo que costaría para una empresa real.

11.1. Coste en el entorno académico

Hardware (clúster Proxmox 3 nodos)	Cedido por el centro	0 €
Software: Proxmox VE, Wazuh, Suricata, Debian, nmap, sqlmap, hydra...	Todo open source	0 €
Tailscale (hasta 3 usuarios)	Plan gratuito	0 €
GitHub (repositorio público)	Plan gratuito	0 €
Tiempo del técnico: ~118 h × 0 €/h (proyecto académico)	—	0 €
TOTAL		0 €

11.2. Estimación para una empresa real

Si una empresa real quisiera replicar este entorno con hardware propio y mano de obra de mercado, el coste sería muy distinto:

Servidor físico para el clúster (3 nodos pequeños o 1 servidor potente)	Una vez	4.000–8.000 €
Software (todo open source, sin cambio)	—	0 €
Tailscale (plan Teams, hasta 25 usuarios)	Anual	~180 €/año
Técnico junior para el montaje inicial (~120 h × 25 €/h)	Una vez	3.000 €
Mantenimiento y monitorización mensual (~8 h × 25 €/h)	Anual	2.400 €/año
Auditoría externa anual (empresa de ciberseguridad)	Anual	3.000–6.000 €
TOTAL AÑO 1 (aprox.)		~13.000 €

Para una gestoría como Apex (10-15 empleados, datos fiscales de clientes), estos costes son asumibles y están muy por debajo del coste de un incidente de seguridad real: una filtración de datos puede suponer sanciones del RGPD de hasta el 4% de la facturación anual, más el daño reputacional.

12. Cumplimiento normativo

La auditoría no solo tiene un valor técnico; también tiene relevancia normativa. Apex Gestoría, al tratar datos personales de sus clientes (NIF, correo electrónico, IBAN, nóminas con DNI y salarios), está sujeta al Reglamento General de Protección de Datos (RGPD).

Los hallazgos de la auditoría tienen implicaciones directas sobre dos artículos clave del RGPD:

- Artículo 25 — Privacidad por diseño y por defecto: la aplicación web fue diseñada sin ningún control de seguridad básico (sin prepared statements, sin validación de ficheros). Esto es exactamente lo contrario de lo que pide el art. 25, que exige incorporar la seguridad desde el diseño.
- Artículo 32 — Seguridad del tratamiento: el art. 32 obliga a implantar medidas técnicas apropiadas al riesgo. Una base de datos con usuario root/toor accesible desde la red, un FTP anónimo con credenciales expuestas y una web con SQLi son el ejemplo perfecto de incumplimiento de este artículo.

La situación más preocupante es la del FTP: el fichero `credenciales.txt` con los IBANs de los clientes y los salarios de los empleados estaba accesible sin autenticación para cualquier persona con internet. Eso es una brecha de datos en el sentido del art. 33, que obliga a notificar a la AEPD en 72 horas si se produce. Si hubiera ocurrido en la realidad, Apex estaría ante una sanción muy probable.

13. Conclusiones

Mirándolo con perspectiva, creo que he cumplido todo lo que me propuse al principio: hay dos infraestructuras de empresa reales y funcionando, un sistema de monitorización que cubre las dos desde un único panel, y una auditoría completa con sus informes. Y, lo que para mí es más importante, todo está hecho de forma que se puede volver a montar, reproducir y demostrar.

La parte técnica que más me ha aportado ha sido el trabajo con nftables y con la segmentación de redes. Antes del proyecto sabía que los firewalls existen y para qué sirven. Después de montarlos a mano, entiendo por qué un DNAT mal configurado o una tabla de reenvío sin filtros puede abrir la puerta a todo. Hay una diferencia enorme entre leer que «hay que segmentar la red» y ver en vivo cómo, sin esa segmentación, un atacante pasa de la web pública a la base de datos de nóminas en dos pasos.

La auditoría me dejó una lección clara sobre la defensa en profundidad: ninguno de los fallos que explotamos por separado era el fin del mundo. La SQLi da acceso a la web. La webshell da ejecución de comandos. La red plana da acceso a la LAN. El FTP anónimo da credenciales. Pero encadenados, el resultado es el acceso completo a todos los datos de la empresa. Si el cliente hubiera tenido bien una sola de esas capas, la cadena se habría roto en algún punto. Eso es lo que significa la defensa en profundidad y es lo primero que le explicaría a Apex en la reunión de presentación de resultados.

En cuanto al SIEM, lo que me satisfizo fue comprobar que detectó el ataque mientras lo hacía. No es perfecto: hay cosas que Wazuh vio y hay cosas que se le escaparon. Pero el escaneo de nmap apareció en Suricata, los intentos de fuerza bruta en Wazuh, la subida de la webshell en los logs de Apache. Un analista con acceso al panel habría podido ver el ataque en tiempo real y actuar. Eso demuestra que el SIEM tiene valor y que vale la pena el esfuerzo de montarlo.

Lo que haría diferente si empezara de cero: le habría dado más tiempo a la fase de diseño de la arquitectura antes de empezar a montar máquinas. Tuve que rehacer algunas cosas porque no había pensado bien cómo iban a hablar los agentes de Wazuh con el servidor, o qué reglas necesitaba el cortafuegos para que cloud-init descargara paquetes al primer arranque. Con un diseño más meditado esos problemas no habrían existido.

Si tuviera más tiempo (y más hardware), lo siguiente que montaría sería Suricata en modo IPS (bloqueo activo) en lugar de solo detección, copias de seguridad automáticas offsite con

verificación, y un sistema de gestión centralizada de identidades (LDAP o Active Directory) para que los usuarios no tengan contraseñas distintas en cada servicio.

En lo personal, lo que más me ha aportado el proyecto ha sido pelearme con los problemas reales que fueron saliendo. Montar las máquinas siguiendo un tutorial es una cosa; entender por qué Suricata se queda sin memoria, por qué el instalador de Wazuh falla en Debian 13 o por qué sqlmap no detecta la inyección sin el flag correcto, y resolverlo, es lo que de verdad me ha hecho aprender. Eso no sale en ningún tutorial.

14. Repositorio y material adicional

Todo el material del proyecto está disponible en el repositorio público de GitHub:

github.com/aleexsanchezz04/jamsec-proyecto-ciberseguridad

El repositorio contiene:

- Los tres informes en PDF: esta memoria, el informe técnico de la auditoría y el informe ejecutivo.
- Los diagramas de infraestructura de JAMSEC y de Apex Gestoría en alta resolución.
- La presentación del proyecto en formato PPTX (13 diapositivas).
- El código fuente de la aplicación web de Apex Gestoría (con las vulnerabilidades intencionales).
- La web corporativa de JAMSEC.
- Las evidencias de la auditoría: capturas del escaneo nmap, volcados de sqlmap, logs de Hydra y el FTP.

Las configuraciones de los cortafuegos (nftables), los scripts de cloud-init y los ficheros de configuración de Wazuh y Suricata no están subidos al repositorio público porque contienen información del entorno (IPs internas, nombres de usuario) que no es apropiado publicar. Están disponibles para revisión por el tribunal si se solicitan.

15. Referencias

- Wazuh Documentation 4.12 — <https://documentation.wazuh.com/current/>
- Suricata User Guide 7.x — <https://suricata.readthedocs.io/>
- Proxmox VE Documentation — <https://pve.proxmox.com/wiki/>
- OWASP Top 10 (2021) — <https://owasp.org/Top10/>
- PTES — Penetration Testing Execution Standard — <http://www.pentest-standard.org/>
- CVSS 3.1 Specification — <https://www.first.org/cvss/>
- RGPD — Reglamento (UE) 2016/679 del Parlamento Europeo
- nftables Wiki — <https://wiki.nftables.org/>
- Tailscale Documentation — <https://tailscale.com/kb/>
- cloud-init Documentation — <https://cloud-init.io/>
- sqlmap User Manual — <https://sqlmap.org/>
- THC Hydra — <https://github.com/vanhauser-thc/thc-hydra>
- Emerging Threats Open Rules — <https://rules.emergingthreats.net/>